Python Interview Reference

Decorators · Dunder Methods · Generators · Context Managers · asyncio

Quick-recall examples for technical interviews

1. Decorators

A decorator wraps a function (or class) to extend behavior without modifying it. It is syntactic sugar for func = decorator(func).

1-A Function Decorator

▸ Pattern: wrapper function returns inner function

```
import functools

def timer(func):                    # decorator factory
    @functools.wraps(func)          # preserve metadata
    def wrapper(*args, **kwargs):
        import time
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.perf_counter()-t0:.4f}s')
        return result
    return wrapper

@timer
def slow_add(a, b):
    import time; time.sleep(0.1)
    return a + b

slow_add(1, 2)   # prints: slow_add took 0.100xs
```

💡 *@functools.wraps copies __name__, __doc__ to wrapper so introspection still works.*

Decorator with arguments (factory pattern)

```
def repeat(n):                      # outer: receives args
    def decorator(func):            # middle: receives func
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator                # return the real decorator

@repeat(3)
def greet(name):
    print(f'Hello {name}')
```

```
greet('Li')   # prints Hello Li  three times
```

1-B Class-Based Decorator

▸ Implement __call__ — the instance becomes the wrapper

```
class Retry:
    def __init__(self, times=3):
        self.times = times

    def __call__(self, func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, self.times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f'Attempt {attempt} failed: {e}')
            raise RuntimeError('All retries exhausted')
        return wrapper

@Retry(times=3)
def flaky():
    import random
    if random.random() < 0.7: raise ValueError('oops')
    return 'ok'
```

💡 *Class decorators are useful when you need to store state across calls (e.g. call count, cache).*

Concept	One-liner
@decorator	func = decorator(func)
@functools.wraps	preserve __name__, __doc__
Decorator with args	3-layer nesting: outer(args) → decorator(func) → wrapper
Class decorator	implement __call__(self, func)
Stacking decorators	@A @B ≡ func = A(B(func)) — B applied first

2. Class Dunder (Magic) Methods

Dunder methods let your objects hook into Python's built-in protocols (arithmetic, comparison, containers, iteration, context managers, etc.).

Core Dunders — Quick Reference

Dunder	Triggered by
__init__	MyClass() — initializer
__repr__	repr(obj), interactive shell
__str__	str(obj), print()
__len__	len(obj)
__getitem__	obj[key]
__setitem__	obj[key] = val
__contains__	x in obj
__iter__	for x in obj, iter(obj)
__next__	next(obj) — iterator protocol
__enter__ / __exit__	with obj:
__eq__ / __lt__	==, < (enable sorting)
__add__	obj + other
__call__	obj() — callable object

Example: a simple Vector class

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f'Vector({self.x}, {self.y})'

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __abs__(self):           # abs(v)
```

```
        return (self.x**2 + self.y**2) ** 0.5

    def __len__(self):           # len(v) → int
        return 2

    def __bool__(self):          # bool(v)
        return bool(abs(self))

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)   # Vector(4, 6)
print(abs(v1))   # 2.236...
```

Example: custom container with __getitem__

```
class RingBuffer:
    def __init__(self, capacity):
        self._buf = [None] * capacity
        self._cap = capacity
        self._head = 0

    def append(self, val):
        self._buf[self._head % self._cap] = val
        self._head += 1

    def __getitem__(self, idx):         # buf[i]
        return self._buf[idx % self._cap]

    def __len__(self):                  # len(buf)
        return min(self._head, self._cap)

    def __repr__(self):
        return f'RingBuffer({self._buf})'
```

💡 *If you define __eq__, Python sets __hash__ = None. Add __hash__ explicitly for hashable objects.*

3. Generators

A generator is a function that yields values lazily. Each call to next() resumes after the last yield. State is preserved automatically.

3-A Generator Function

```
def countdown(n):
    while n > 0:
        yield n          # suspend here, return n to caller
        n -= 1           # resumes here on next()

for x in countdown(3):  # 3  2  1
    print(x)

# Manual iteration
gen = countdown(2)
print(next(gen))   # 2
print(next(gen))   # 1
# next(gen) → StopIteration
```

3-B Infinite Generator

```
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

import itertools
first10 = list(itertools.islice(fib(), 10))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

3-C Generator Expression

▸ Like list comprehension but lazy — no brackets

```
squares = (x*x for x in range(1_000_000))  # O(1) memory

total = sum(x*x for x in range(1000))      # pipeline

# vs list comprehension (eager, loads all into memory)
squares_list = [x*x for x in range(1_000_000)]  # O(n) memory
```

3-D send() and yield as expression (coroutine-style)

```
def accumulator():
    total = 0
    while True:
        value = yield total   # yield sends total OUT; receives value IN
        if value is None:
            break
        total += value

acc = accumulator()
next(acc)          # prime: advance to first yield
print(acc.send(10))  # 10
print(acc.send(5))   # 15
print(acc.send(3))   # 18
```

Feature	Notes
yield	pause function, return value to caller
next(gen)	resume from last yield; StopIteration when done
gen.send(val)	resume AND pass val in as yield expression result
gen.throw(exc)	raise exception inside generator at yield point
gen.close()	throw GeneratorExit to cleanly shut down
Generator expr	(x for x in ...) — lazy, O(1) memory
yield from sub()	delegate to sub-generator; passes send/throw through

💡 *Generators implement __iter__ and __next__ automatically — they ARE iterators.*

4. Context Managers

Context managers guarantee setup and teardown via the `with` statement. Python calls `__enter__` on entry and `__exit__` on exit (even on exception).

4-A Class-Based Context Manager

```
class ManagedFile:
    def __init__(self, path, mode='r'):
        self.path, self.mode = path, mode

    def __enter__(self):
        self.file = open(self.path, self.mode)
        return self.file          # bound to  as  target

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False              # False = don't suppress exceptions

with ManagedFile('data.txt') as f:
    content = f.read()
# file is closed here, even if an exception was raised
```

💡 *Return True from __exit__ to suppress the exception. Usually return False.*

4-B Generator-Based with @contextmanager

▸ **Everything before yield = __enter__; after yield = __exit__**

```
from contextlib import contextmanager

@contextmanager
def timer_ctx(label):
    import time
    t0 = time.perf_counter()            # __enter__ logic
    try:
        yield                           # 'as' target = None here
    finally:                            # __exit__ logic
        elapsed = time.perf_counter() - t0
        print(f'{label}: {elapsed:.4f}s')

with timer_ctx('matrix multiply'):
    result = [i*i for i in range(10_000)]
# prints: matrix multiply: 0.0012s
```

4-C Nested & Multiple Context Managers

```
# Python 3.10+ parenthesised form
with (
    open('input.txt')  as src,
    open('output.txt', 'w') as dst,
):
    dst.write(src.read().upper())

# Classic (any version)
with open('a') as a, open('b') as b:
    pass
```

4-D contextlib.suppress — swallow specific exceptions

```
from contextlib import suppress

with suppress(FileNotFoundError):
    os.remove('maybe_exists.tmp')   # no try/except needed
```

Method / decorator	Purpose
__enter__(self)	setup; return value bound to as variable
__exit__(exc_t,v,tb)	teardown; return True to suppress exception
@contextmanager	turn generator into context manager (yield once)
contextlib.suppress	context manager that swallows given exception types
contextlib.ExitStack	dynamically compose N context managers

5. asyncio

asyncio enables concurrent I/O-bound code in a single thread using cooperative multitasking. Key primitives: coroutines (async def), Tasks, and the event loop.

5-A Coroutine basics

```
import asyncio

async def fetch(url):              # coroutine function
    print(f'fetching {url}')
    await asyncio.sleep(1)         # yield control back to event loop
    return f'data from {url}'

async def main():
    result = await fetch('api/v1') # await suspends until done
    print(result)

asyncio.run(main())   # entry point — creates & runs event loop
```

💡 *await can only appear inside async def. asyncio.sleep() never blocks the thread.*

5-B Concurrent tasks with asyncio.gather

▸ **gather runs coroutines concurrently; total time ≈ max(individual times)**

```
async def fetch(url, delay):
    await asyncio.sleep(delay)
    return f'{url} done'

async def main():
    # All three run concurrently — total ~1s, not 1+2+3=6s
    results = await asyncio.gather(
        fetch('api/users',    delay=1),
        fetch('api/orders',   delay=2),
        fetch('api/products', delay=3),
    )
    for r in results:
        print(r)

asyncio.run(main())
```

5-C asyncio.create_task — fire and don't block

```
async def background_job(name):
    await asyncio.sleep(2)
    print(f'{name} done')

async def main():
    task = asyncio.create_task(background_job('sync-db'))  # starts now
    print('doing other work...')
    await asyncio.sleep(0.5)         # yields; task runs in background
    print('still doing work...')
    await task                        # wait for task to finish

asyncio.run(main())
# Output: doing other work... → still doing work... → sync-db done
```

5-D async for / async with

```
# async with — context manager that can await
async def download(url):
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as resp:
            return await resp.text()

# async for — async generator iteration
async def stream_lines(path):
    async with aiofiles.open(path) as f:
        async for line in f:
            yield line.strip()

async def main():
    async for line in stream_lines('big.log'):
        process(line)
```

5-E asyncio.Queue — producer / consumer

```
async def producer(queue, items):
    for item in items:
        await queue.put(item)
        await asyncio.sleep(0.1)    # simulate I/O
    await queue.put(None)           # sentinel

async def consumer(queue):
    while True:
```

```
        item = await queue.get()
        if item is None:
            break
        print(f'processed: {item}')
        queue.task_done()

async def main():
    q = asyncio.Queue(maxsize=5)
    await asyncio.gather(
        producer(q, range(10)),
        consumer(q),
    )

asyncio.run(main())
```

Concept	One-liner
async def f()	defines a coroutine; calling it returns a coroutine object
await expr	suspend current coroutine until expr is complete
asyncio.run(coro)	create event loop, run coro, close loop
asyncio.gather(*cos)	run coroutines concurrently, return list of results
asyncio.create_task	schedule coroutine without awaiting immediately
async with	context manager whose __aenter__/__aexit__ are async
async for	iterate over async generator (implements __aiter__)
asyncio.Queue	thread-safe async queue for producer/consumer patterns
asyncio.sleep(0)	yield control to event loop without sleeping (cooperative yield)

💡 *asyncio is single-threaded. Use concurrent.futures.ThreadPoolExecutor for CPU-bound work inside async code: await loop.run_in_executor(executor, blocking_fn).*

Quick-Recall Cheat Sheet

Topic	Key Mental Model
Function decorator	func = wrapper(func) — 3-layer if it takes args
Class decorator	implement __call__(self, func) to make it wrappable
@functools.wraps	always use it to preserve metadata
__repr__ vs __str__	repr = unambiguous (dev), str = readable (user)
__enter__/__exit__	setup/teardown; __exit__ gets exc info; True = suppress
yield in generator	suspends, returns value; next() resumes
send(val) to generator	resume AND inject value; prime with next() first
@contextmanager	yield once: before=enter, after(finally)=exit
asyncio.gather	concurrent tasks; total time ≈ slowest, not sum
create_task vs gather	task = fire-and-forget; gather = wait-for-all
async for/with	use __aiter__/__anext__ and __aenter__/__aexit__
asyncio CPU block	run_in_executor to offload blocking/CPU work